

Cahier des charges — Mini-projet AMC 2026

Voice Recorder : enregistrement, transcription et résumé de discussions

Guilhem Rorier Vilardell¹

Haute école de paysagerie, d'ingénierie et d'architecture, Genève, Suisse

Type de projet : Monôme
Cours : Application Mobile avec Capteurs (AMC)
Année : 2026

1. Contexte et objectif

Application Python-Kivy permettant d'enregistrer une discussion via le **microphone** de l'appareil (plyer.audio sur Android, speech_recognition/pyaudio sur Windows), de transcrire l'audio en texte, d'en générer un résumé et d'exporter le résultat en .md. Cas d'usage : prise de notes passive lors de réunions ou cours. L'application est fonctionnelle en **simulation Windows** et **déployable sur tablette Android** via Buildozer (.apk).

2. Fonctionnalités principales

- **Enregistrement audio** : démarrage/arrêt via boutons, capture en .wav stocké temporairement.
- **Transcription** : envoi à Google Speech-to-Text via speech_recognition, en français, connexion requise.
- **Résumé automatique** : appel à l'API **Google Gemini 2.5 Flash** (prompt structuré), clé API dans config.env.
- **Export Markdown** : transcription + résumé sauvegardés en .md et affichés dans l'interface.
- **Historique SQLite** : 10 derniers enregistrements (date, durée, extrait) consultables depuis un écran dédié.
- **Copier-coller** : bouton pour copier le .md affiché dans le presse-papier.

3. Fonctionnalités non abordées en cours

- **API de transcription vocale** : speech_recognition avec backend Google Speech-to-Text [1].
- **LLM via API REST** : intégration de Google Gemini pour la génération de résumés en langage naturel [2].
- **SQLite embarqué** : historique persistant avec requêtes SQL dans une application Kivy [3].
- **Fichiers Markdown** : écriture, lecture et affichage de .md avec accès au système de fichiers Android.

4. Compatibilité et déploiement

Windows : exécution directe avec Python et micro du PC. **Android** : .apk généré via Buildozer [4], permissions RECORD_AUDIO, INTERNET et WRITE_EXTERNAL_STORAGE déclarées dans buildozer.spec. La clé API est stockée dans config.env (exclu du dépôt via .gitignore).

5. Répartition des tâches

TABLE 1 : Liste des tâches du projet

Priorité	Tâche
1	Architecture et interface graphique Kivy (écrans, navigation).
2	Accès au microphone (plyer.audio Android, speech_recognition Windows).
3	Transcription via Google Speech-to-Text.
4	Résumé via Google Gemini 2.5 Flash.
5	Export et affichage du fichier .md.
6	Base de données SQLite pour l'historique (10 entrées).
7	Bouton de copie dans le presse-papier.
8	Tests et débogage sur Windows (mode simulation).
9	Génération du .apk via Buildozer et tests sur tablette Android.
10	Rédaction du README.md et mise à jour du cahier des charges.
11	GitEdu et JOURNAL.md.
12	Vidéo de démonstration (SWITCHtube).

1. MT2, guilhem.rorier-vilardell@hes-so.ch

6. Références

Références

- [1] SpeechRecognition Library, *SpeechRecognition 3.x — Python speech recognition library*, GitHub, 2024. https://github.com/Uberi/speech_recognition (consulté le 29 avril 2026).
- [2] Google, *Gemini API documentation — Google AI Studio*, Google Developers, 2025. <https://ai.google.dev/gemini-api/docs> (consulté le 29 avril 2026).
- [3] SQLite Consortium, *SQLite Documentation*, 2024. <https://www.sqlite.org/docs.html> (consulté le 29 avril 2026).
- [4] Kivy Organization, *Buildozer documentation — Package a Kivy application for Android*, Read the Docs, 2024. <https://buildozer.readthedocs.io/en/latest/> (consulté le 29 avril 2026).